

5

Combinatorial Optimization

5.1. Introduction

The term “combinatorial optimization” encompasses any optimization problem involving binary variables, together with appropriate techniques for solving such problems. The objective of combinatorial optimization is to choose a configuration that satisfies the conditions from the set of possible configurations for a given problem, while optimizing certain criteria. These criteria are generally represented as a function, known as the cost function or objective function.

Currently, there is growing pressure to not only find a solution that satisfies the criteria and is reasonably efficient, but also to find one that is in fact the best possible solution. This is the case for decision problems in production management, logistics and telecommunications, where size increases rapidly and the financial stakes are huge.

In its most general form, a combinatorial optimization (or discrete optimization) problem finds the best feasible subsets (or solutions) from a discrete set. The concept of best solution is defined in terms of an objective function. Formally, given:

- a discrete set N ;
- a set function $f: 2^N \rightarrow \mathbb{R}$, called the objective function;
- and a set $\mathcal{R}$ of subsets of N whose elements are called feasible solutions;

the goal of a combinatorial optimization problem is to determine:

$$\max_{S \in \mathcal{R}} \{f(S) : S \subseteq N\}.$$

The set of feasible solutions cannot be listed exhaustively. Indeed, the difficulty of combinatorial optimization problems is that the sheer number of feasible solutions makes them impossible to enumerate, and even determining whether feasible solutions exist may be an NP-complete problem. The description of $\mathcal{R}$ is therefore stated implicitly. In general, this is the relatively short list of the properties of feasible solutions.

Most of the time, we are faced with one of the following special cases:

- $N = \{1, 2, \dots, n\}$ $N = \{1, 2, \dots, n\}$ and $\mathcal{R} = X \cap \mathbb{Z}^n$, where $X \subset \mathbb{R}^n$ and the cardinality of $\mathcal{R}$ is exponential with respect to n ;
- $\mathcal{R} = \text{conv}(\mathcal{R}) \cap \mathbb{Z}^n$, so the properties of $\mathcal{R}$ translate to linear constraints, i.e. X is a polyhedron on $\mathbb{R}^n$;
- $f(S) = \sum_{s \in S} f(s)$.

A non-exhaustive list of combinatorial optimization problems might include the following:

- Loading problem: given a collection of objects, each characterized by a volume and a value, and a bag of limited volume, which objects should we load into the bag to maximize the total loading value?
- Traveling salesman problem (TSP): given n towns and the distances between them, which path should we take to visit every town exactly once while traveling the minimum possible distance?
- Covering problem: given a set of objects and a set of subsets of this set, each with a certain cost, which subsets should we choose to guarantee that every object is taken at least once while minimizing the total cost?
- Vehicle routing problem (VRP): given a set of clients that must each be visited exactly once to fulfill a certain demand, a fleet of delivery vehicles of limited capacity parked at the warehouse and the distances between each pair of clients and the distance between each client and the warehouse, which delivery routes should be followed in order to satisfy the demand of every client while minimizing the total distance traveled?
- Localization problem: there are several versions of this problem. One simple version can be stated as follows: if the goal is to minimize the total cost, how many warehouses should be opened and where should they be located given that (i) there are multiple different possible sites for opening a warehouse, each with a certain opening cost, and (ii) the warehouses must serve a set of clients, incurring costs that depend on which warehouse serves which client?

Combinatorial optimization is becoming increasingly important. From operations research to artificial intelligence, various solving methods have been proposed. These methods can essentially be classified into two families. The first family contains so-called exact, complete or optimal methods that guarantee admissible and optimal solutions, with respect to the function

defined for the relevant problem. If such a solution does not exist, these methods must be able to conclude that there is no solution. The second family, known as approximate, incomplete or heuristic methods, has been proposed to mitigate combinatorial explosion at the expense of no longer guaranteeing optimality.

Complete methods use tree-based searches. They start from an empty solution and construct the optimal solution (if it exists) by ranging over the set of variables and instantiating them one by one. The approach taken by these methods can therefore be described as constructive.

Backtracking is often necessary if the constraints are violated, which makes the search more complex. Despite much work on expanding the search tree, detecting inconsistencies at an earlier stage and/or reducing the amount of backtracking with heuristics, such methods remain exponential, meaning that they explode as the size of the problem increases.

Approximate methods, also known as heuristic methods, begin with a complete initial solution S_0 , i.e. a solution where every variable has been instantiated. They then repair this solution step by step until a predefined stopping criterion is reached; the goal is of course to find a solution S^* which one hopes to be optimal with respect to the cost. These methods use an approach that can be described as local, neighboring or repair-based.

This chapter presents two very well-known problems of combinatorial optimization: the TSP and the VRP. The reader can refer to [TEG 12, GHE 07, KOR 11] for more details.

5.2. Symmetric TSP

Human beings face various problems across every part of society. A problem can be defined in terms of a set of properties that must be satisfied by its solutions. This might be a decision problem or an optimization problem.

In practice, in an optimization problem it is often desirable for some of its variables to be integers or even binary, $x_j = 0$ or 1 . This is described as a discrete integer linear program (ILP), or a binary linear program in the binary case. One of the most widely studied problems in the class of ILP is the TSP, which was posed for the very first time by Euler (1736). Since then, it has been widely studied in the literature in its classical form. In 1859, it was stated as follows: a traveling salesman must visit finitely many towns exactly once, then return to his starting point. Find the order in which the towns should be visited in order to minimize the total distance traveled.

To better understand the problem, we can say that the traveling salesman should visit n given towns while passing through each town exactly once. He starts with an arbitrary town and ends by returning to the town in which he started. The distances between towns are known. Which path should he take to minimize the distance traveled? The notion of distance can also be replaced by other notions, such as the time or money that he must spend; in general, we can speak of a cost (see [Figure 5.5](#)).

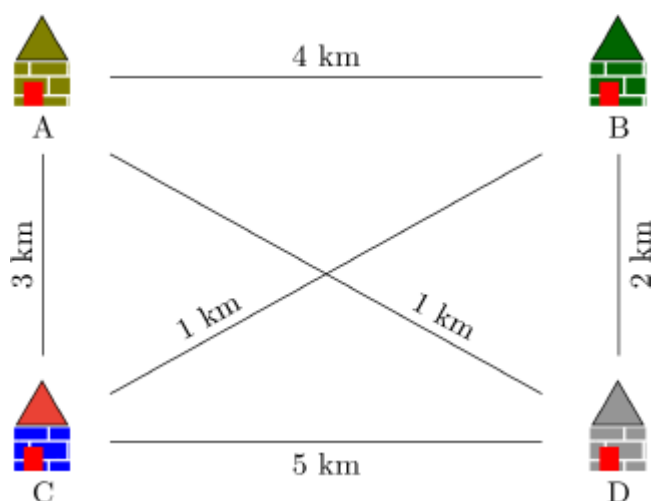


Figure 5.1. Different possible trips. For a color version of this figure, see www.iste.co.uk/radi/optimizations.zip

Which path is the shortest if the salesman starts at A, passes through every other point exactly once, then returns to A?

5.2.1. Historical overview

The origin of the name of the “traveling salesman problem” is not known exactly. It is one of the oldest combinatorial problems. William Rowan Hamilton made it into a game: Hamilton’s Icosian game. Players had to choose a route passing through 20 points using only the predefined connections. In 1930, the TSP was studied in more depth by Karl Menger at Harvard, before being further developed at Princeton by the mathematicians Hassler Whitney and Merrill Flood. Thus, a variety of mathematicians have taken up this problem since the early 20th century in search of a scientific answer to this very real question.

Tolga Bekta proposed several ways to solve the TSP [BEK 06]. In 1998, Arora [ARO 98] showed, based on a proof by Papadimitriou [PAP 77], that the Euclidean TSP belongs to the class of NP-hard optimization problems, which have a computational complexity that increases exponentially with respect to the number of towns in the problem. Dantzig *et al.* [DAN 54] solved

a TSP problem with 49 towns using the cut generation method. Padberg and Rinaldi [PAD 91] solved a problem with 532 towns, followed by another problem with 2392 towns. In 2006, Applegate *et al.* [APP 06] solved a problem with 85,900 towns (see [Figure 5.2](#)). The renowned Concorde program [APP 06] for solving the symmetric TSP has been the best-performing algorithm for the past 20 years. The program uses the branch-and-cut method, generating constraints from valid inequalities that have been specifically studied because of the TSP.

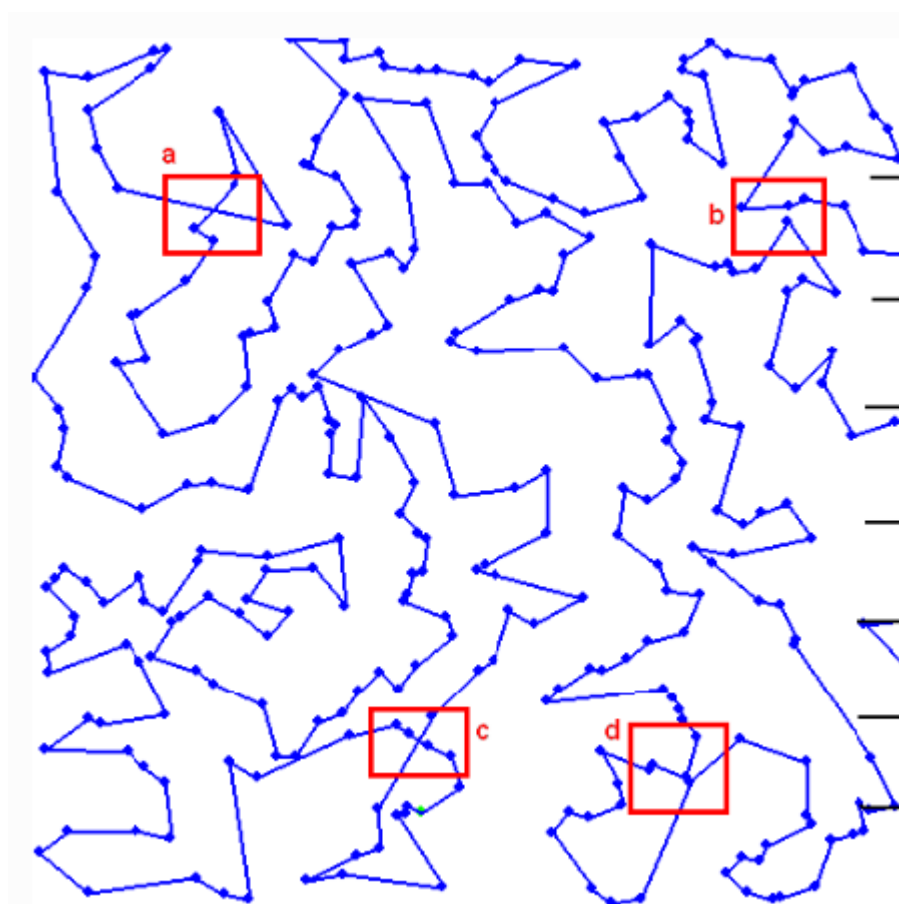


Figure 5.2. Problem with 85,900 towns. For a color version of this figure, see www.iste.co.uk/radi/optimizations.zip

The TSP can be divided into two types, each with several different variants: the symmetric traveling salesman problem (STSP) and the asymmetric traveling salesman problem (ATSP). This chapter will present the asymmetric case. In the ATSP, $d_{ij} \neq d_{ji}$, whereas in the STSP, $d_{ij} = d_{ji}$, where d_{ij} is the distance between two nodes i and j .

This problem has many practical applications and serves as a reference for other problems, particularly from the fields of transportation and logistics. Examples include finding the shortest path for a schoolbus to pick children up or the shortest distance traveled by the mechanical arm of

a machine when punching holes into a circuitboard in an industrial setting. Many variants of the TSP can be obtained either by adding constraints – such as the probabilistic traveling salesman problem (PTSP) – or by modifying the problem, e.g. the VRP.

5.2.2. Solving methods

5.2.2.1. Exact methods

- Branch-and-bound: this method is based on “intelligently” enumerating the admissible solutions of a combinatorial optimization problem that involves finding a point that minimizes or maximizes a function, subject to several constraints. It is widely used to solve problems that are considered difficult to solve efficiently.
- Little algorithm: when the problem being solved involves binary variables, this algorithm can be used to simplify the computations. It is a simplification of the branch-and-bound algorithm (Algorithm 5.1).

Algorithm 5.1 Little algorithm

- 1: Reduce the matrix (row and column)
- 2: Select the empty cell that is most expensive to eliminate.
- 3: The elimination cost is the minimum extra cost incurred if the cell is not selected, i.e. if another cell on the same row and another cell on the same column are selected.
- 4: Remove the row of the starting town and the column of the destination town.
- 5: Set the return cost to infinity.
- 6: Restart with the subtable.
- 7: Present the algorithm as a tree (branch-and-bound).
- 8: Consider any path with lower cost than the first final cost that we find.
- 9: Follow the tree until it terminates.

5.2.2.2. Approximate methods

- Greedy algorithm: follows the principle of making a locally optimal choice step by step in the hope of obtaining a globally optimal result. For example, in the change problem (giving

change with the fewest possible coins), the greedy algorithm simply chooses the biggest coin that does not exceed the remaining change to be given. In cases where the algorithm does not always find the optimal solution, it is called a greedy heuristic.

- Methods of descent: involves moving through the search space by always choosing the best solution among the neighbors of the current solution.

- Simulated annealing: an empirical (metaheuristic) method inspired by a process used in metallurgy that alternates between slow cooling cycles and heating cycles (annealing) to minimize the energy of the material. This method can be transposed to optimization to find the extrema of a function.

- Tabu search: an optimization metaheuristic. The idea is to explore the neighborhood around a given position and choose the position in this neighborhood that minimizes the objective function.

- Genetic algorithm: based on the principle of selecting individuals (routes) from a population (set of routes) with properties as close as possible to the desired properties. A selection criterion needs to be defined, as well as rules describing the evolution of the population. However, there are very few theoretical results involving genetic algorithms, and the choice of criteria and rules is therefore heuristic.

- Lin and Kernighan algorithm: one of the best heuristics for the TSP. The algorithm exchanges a certain number of edges from a given solution to find a solution with a better cost. This method exchanges two or three edges to make the route shorter.

- Ant colony algorithms: a family of metaheuristic optimization algorithms based on the behavior of ants and other species that form superorganisms. Proposed to find an optimal path in a graph, the first algorithm was inspired by the behavior of ants looking for a path between their colony and a food source.

[Figure 5.3](#) summarizes the various solving methods cited above.

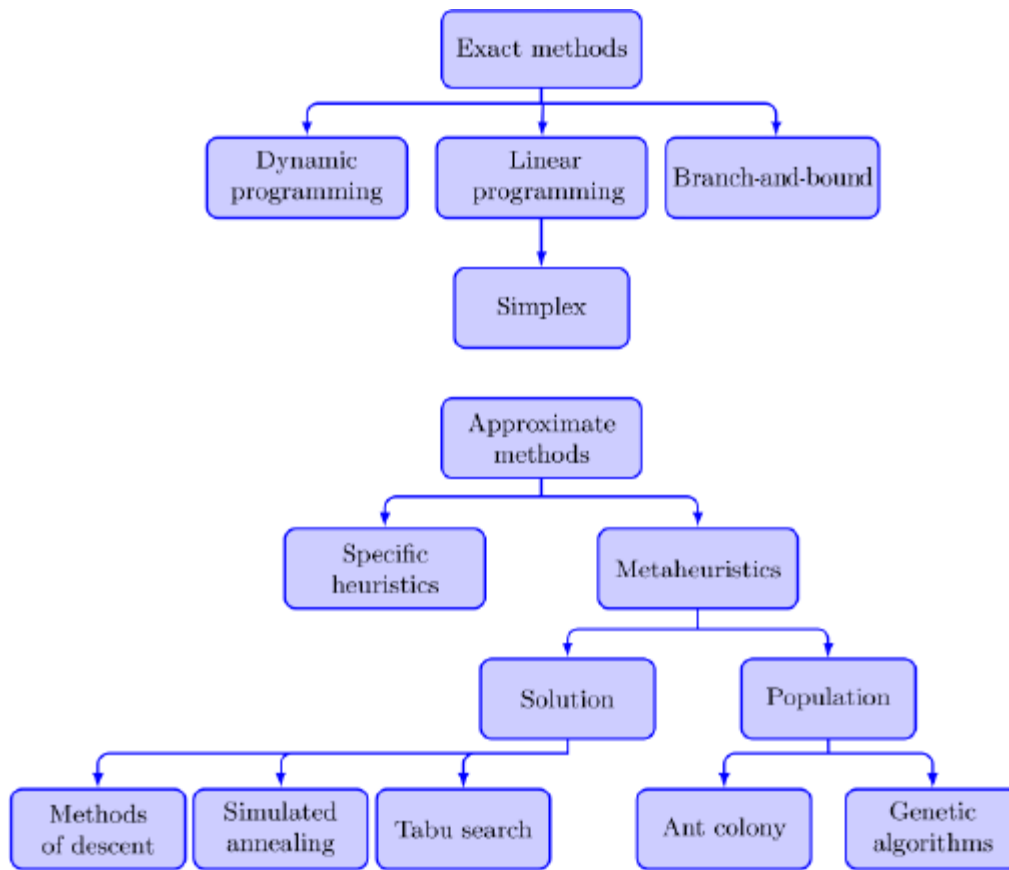


Figure 5.3. *Various solving methods*

Consider the following table listing transportation costs between towns:

Starting towns	Destinations						Min
	A	B	C	D	E	F	
A	-	1	7	3	14	2	1
B	3	-	6	9	1	24	1
C	6	14	-	3	7	3	3
D	2	3	5	-	9	11	2
E	15	7	11	2	-	4	2
F	20	5	13	4	18	-	4

The first step of the method is to reduce the matrix by subtracting the smallest element from each row. This gives the reduced matrix illustrated in the following table:

Starting towns	Destinations					
	A	B	C	D	E	F
A	-	0	6	2	13	1
B	2	-	5	8	0	23
C	3	11	-	0	4	0
D	0	1	3	-	7	9
E	13	5	9	0	-	2
F	16	1	9	0	14	-
Min	0	0	3	0	0	0

The table is easy to interpret: each transportation cost between town i and town j has been replaced by the difference between this transportation cost and the lowest transportation cost required to leave the town. The sum of the minimum costs is $1 + 1 + 3 + 2 + 2 + 4 = 13$.

Starting towns	Destinations					
	A	B	C	D	E	F
A	-	0	3	2	13	1
B	2	-	2	8	0	23
C	3	11	-	0	4	0
D	0	1	0	-	7	9
E	13	5	6	0	-	2
F	16	1	6	0	14	-

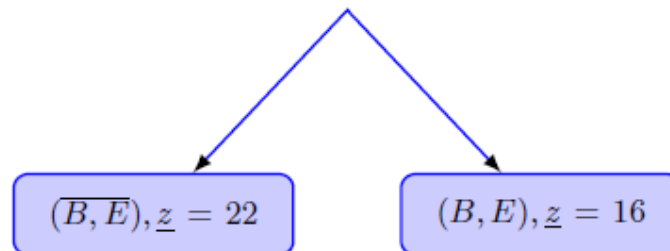
This matrix can be interpreted as the extra cost of arriving in each town relative to the minimum cost of arriving, or alternatively as the table of transportation costs that need to be added to the minimum cost of $13 + 3 = 16$. To choose the first pair, we need to compute the penalties as follows: $a_{ij} = \min c_{ik} + \min c_{hj}$. The next table recalculates these penalties for each pair without the extra cost.

Starting towns	Destinations					
	A	B	C	D	E	F
A	-	0(2)	3	2	13	1
B	2	-	2	8	0(6)	23
C	3	11	-	0(0)	4	0

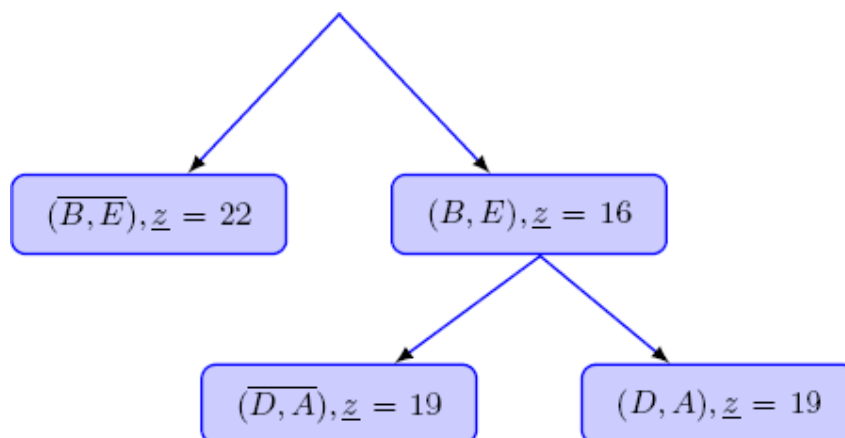
D	0(2)	1	0(2)	-	7	9
E	13	5	6	0(2)	-	2
F	16	1	6	0(1)	14	-

To choose an edge, we will take the pair B–E, because it has the largest penalty. Therefore, we need to delete row B and column E to go from a problem with six towns to a problem with five. This requires us to reduce the matrix and recalculate the penalties once again.

Starting towns	Destinations				
	A	B	C	D	F
A	-	0(2)	3	2	1
C	3	11	-	0(0)	0
D	0(2)	1	0(2)	-	9
E	13	5	6	0(2)	2
F	16	1	6	0(1)	-



We will determine the pair (i, j) for which the new penalty is highest. There are two of them. As an example, we will branch at (D, A) as follows: $z = 16 + 3 = 19$.



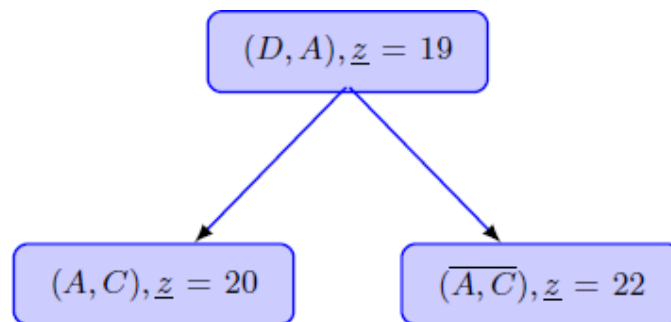
Repeating the same steps gives:

--	--

Starting towns	Destinations			
	B	C	D	F
A	0	3	-	1
C	11	-	0	0
E	-	6	0	2
F	1	6	0	-

After computing, we select the pair A, C. The minimum cost is therefore: $z = 19 + 3 = 22$.

Starting towns	Destinations			
	B	C	D	F
A	0(1)	0(3)	-	1
C	11	-	0(0)	0(1)
E	-	3	0(2)	2
F	1	3	0(1)	-



Next, we obtain the matrix that is shown in the following table:

Starting towns	Destinations		
	B	D	F
C	10	-	0
E	-	0	2
F	0	0	-

The minimum cost associated with this choice is therefore: $z = 19 + 1 = 20$.

Starting towns	Destinations		

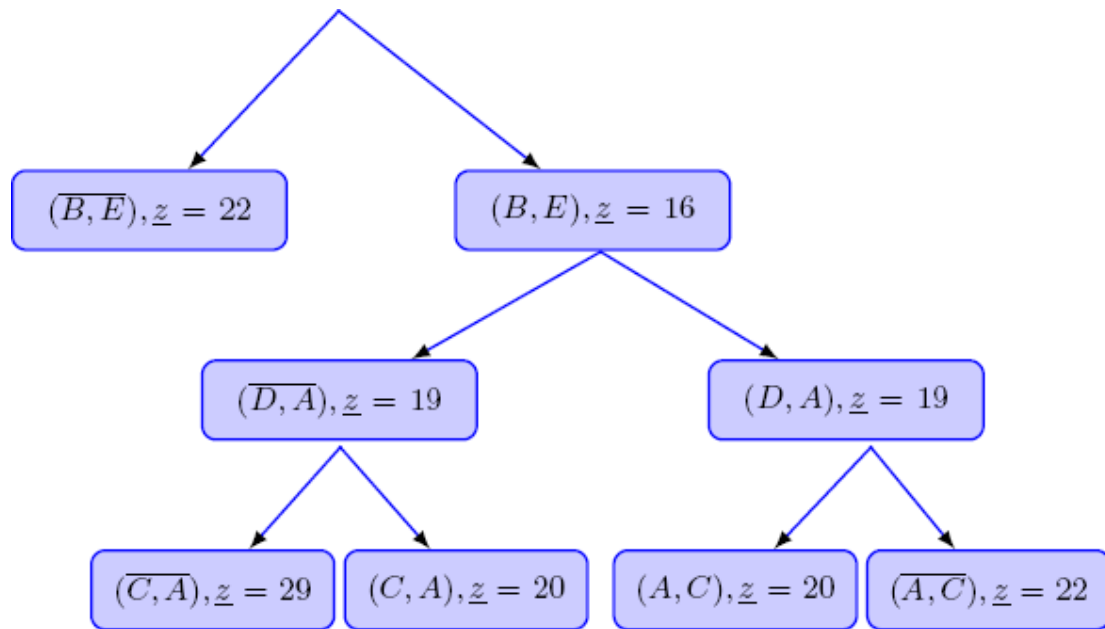
	B	D	F
C	11	-	0
E	-	0	2
F	1	0	-

We reform the matrix by taking the last table into account:

Starting towns	Destinations					
	A	B	C	D	F	
A	-	0	3	2	1	
C	3	11	-	0	0	
D	-	1	0	-	9	
E	13	-	6	0	2	
F	16	1	6	0	-	

Now we need to reduce the matrix using the following table:

Starting towns	Destinations					
	A	B	C	D	F	
A	-	0(2)	3	2	1	
C	0(10)	11	-	0(0)	0(1)	
D	-	1	0(4)	-	9	
E	10	-	6	0(2)	2	
F	13	1	6	0(1)	-	



Finally, we need to eliminate the edge (A, C) , then continue in the same way. We ultimately find that the optimal route is as follows: A, C, F, B, E, D with $z = 20$.

5.3. Asymmetric traveling salesman problem

The ATSP is defined on a directed graph $G = (V, E)$, where $V = \{1, \dots, n\}$ represents the set of nodes (vertices), $E = \{(i, j) : i, j \in V\}$ is the set of edges, and the non-symmetric cost matrix c_{ij} is defined on E . Solving the ATSP means finding a Hamiltonian cycle on G with the lowest cost. The problem is often interpreted as finding the optimal route of a traveling salesman through n towns. The ATSP is NP-hard even when the costs are Euclidean (see [Figure 5.4](#)).

5.3.1. Variants of the ATSP

Several variants of the ATSP can be formulated to describe it including:

- the periodic ATSP: in this generalization of the ATSP, we define a schedule over a given period of k days. Thus, we want to find k cycles in G such that each node of G is visited a certain number of times over the k -day period and the total cost of the route over the k -day period is minimal [PAL 04];
- ATSP with replenishment arcs (RATSP): the goal is to find a route of minimal cost. In this context, we allow two parallel edges to be defined between the same two nodes, namely a replenishment edge and an ordinary edge, provided that the ordinary edge has lower cost. The RATSP is a generalization of the ATSP with various applications in routing problems. For example, it is used in the aircraft routing problem, where maintenance options are

modeled as replenishment arcs, as well as the VRP, where customer demand and vehicle capacity can be modeled as replenishment arcs [BOL 00];

– precedence-constrained ATSP: also known as the sequential ordering problem (SOP), the precedence-constrained ATSP requires certain nodes to be preceded by others in any feasible route. This problem is an essential model in scheduling and routing and has a wide range of applications, including vehicle routing, sequencing in manufacturing industries, and so on [SAR 05, GOU 06];

– ATSP with time windows (ATSP-TW): this is a classical scheduling and demand pathing problem. The idea is to impose time constraints specifying when each vertex may be visited [ASC 01];

– ATSP with jump constraints (Hamiltonian TSP–HTSP): let S be the set of towns subject to the jump constraint (i.e. whose distance in terms of the number of edges between towns is at least s). The HTSP searches for a Hamiltonian cycle of minimal cost that observes the distance constraints between the towns of S ;

– probabilistic ATSP with clients: here, we assume that each client is visited with probability p_i , independently of the others (variant of the classical ATSP).

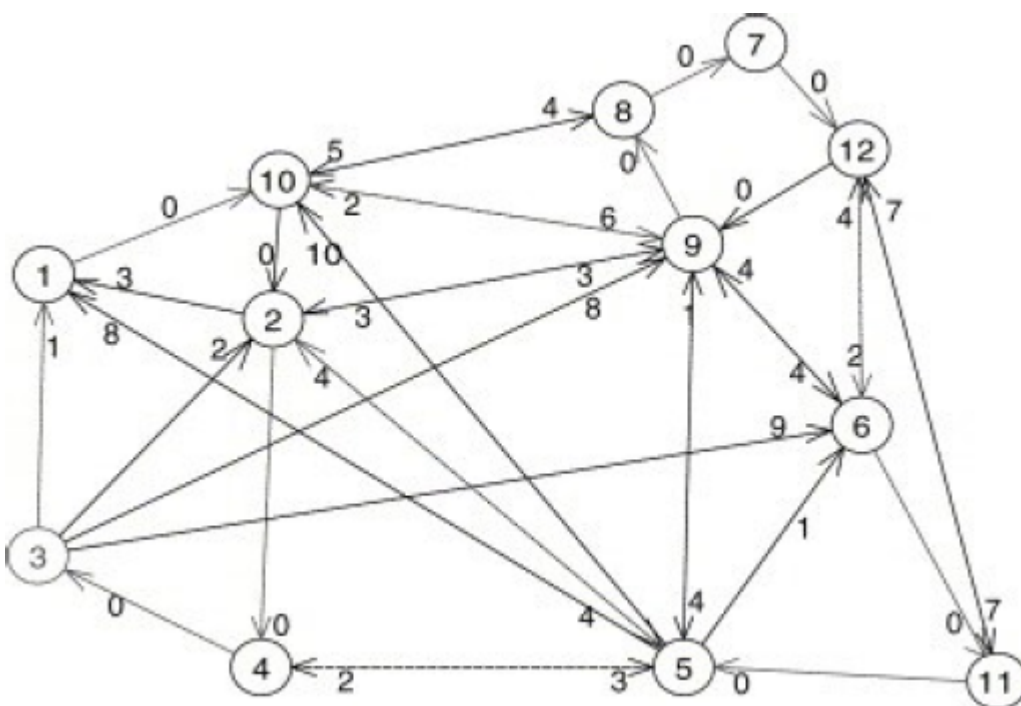


Figure 5.4. *Traveling salesman's route through n towns*

In general, the ATSP is known to be an NP-hard problem. It has been studied intensively over the past six decades. Early studies on the topic were presented by Fischetti *et al.* [FIS 94] and Oncan

et al. [ÖNC 09]. Several books discussing the TSP and its variants have been published.

Examples include the books by Lawler *et al.* [LAW 85], Reinelt [JÜN 95], Gutin and Punnen [GUT 06] and Applegate *et al.* [APP 06].

5.3.2. Mathematical formulations

Let $G = (V, A)$ be a complete and directed graph, where the set V represents the towns to be visited, including the starting town of the route, and A is the set of edges of G . Each edge in A is assigned a travel distance d_{ij} from town i to town j . The length of a path in G is the sum of the distances of the edges on the path. Thus, the TSP reduces to find a Hamiltonian cycle (closed path passing through every vertex of the graph exactly once) of minimal length in G . In general, the TSP can equivalently be formulated by assigning each pair of towns to be visited ($i = 1$ to n , $j = 1$ to n and $i \neq j$), a binary succession variable that takes the value 1 if town j is visited immediately after town i in the route and 0 otherwise, with the set of successors of i . The basic idea of the mathematical formulation of the TSP is to begin with an assignment problem (AP): each town i is assigned a town j that should succeed it on the salesman's route. This gives the formulation:

$$\min \sum_{i,j} d_{ij} x_{ij} \quad [5.1]$$

$$\text{s.t.} \quad \sum_{j \in N^+(i)} x_{ij} = 1 \quad \forall i \in V \quad [5.2]$$

$$\sum_{i \in N^+(j)} x_{ij} = 1 \quad \forall j \in V \quad [5.3]$$

$$x_{ij} \in \{0, 1\}, \forall (i, j) \in A \quad [5.4]$$

The first two constraints, [5.2] and [5.3], translate the fact that each town must be visited exactly once, and [5.4] is an integrality constraint. This formulation is not yet valid in the sense that a solution does not necessarily give a full cycle: in general, it consists of several cycles (subtours). We therefore need to extend this formulation in such a way that cycles (subtours) are prohibited. There are two ways to do this classically, proposed by Dantzig *et al.* [DAN 54]. Miller *et al.* [MIL 60] rewrote the problem differently to obtain two formulations of the TSP: the DFJ (Dantzig, Fulkerson and Johnson) formulation and the MTZ (Miller, Tucker and Zemlin) formulation. A few years later, Desrochers and Laporte reinforced the MTZ formulation by proposing the DL formulation [DES 91].

5.3.2.1. DFJ formulation

Here, A represents the set of edges (i, j) and V is the set of towns, with n being the number of towns to be visited. The value l_{ij} represents the distance between i and j . We also use a binary variable: for each edge $(i, j) \in A$, the variable x_{ij} is 1 if the edge (i, j) is taken and 0 otherwise, with S the subset of vertices that have been visited (in subtours) [DAN 54]:

$$\min \sum_{i \neq j} d_{ij} x_{ij} \quad [5.5]$$

$$\text{s.t. } \sum_{j=1}^n x_{ij} = 1 \quad (\forall i \in V, i \neq j) \quad [5.6]$$

$$\sum_{i=1}^n x_{ij} = 1 \quad (\forall j \in V, j \neq i) \quad [5.7]$$

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1 \quad (S \subset V, 2 \leq |S| \leq n - 2) \quad [5.8]$$

$$x_{ij} \in \{0; 1\}, \quad \forall (i, j) \in A \quad [5.9]$$

The constraint [5.5] represents the objective function, [5.6] and [5.7] represent the assignment constraints, [5.8] represents the subtour elimination constraints and [5.9] is the integrality constraint.

This formulation proves to be difficult to manipulate in practice due to the size of the model as the number of towns increases. This increase is linked, in particular, to the exponential number of subtour elimination constraints that are generated. This formulation of the TSP contains $n(n - 1)$ variables and $(2^{n-1} + n - 2)$ constraints.

Although it is easy to express, the primary shortcoming of this method is its size, which quickly becomes prohibitively large. Indeed, for an n -dimensional problem, there is a very large number of subtour elimination constraints, which quickly renders problems unsolvable as the number of clients to be visited increases. Alternative methods that have been developed therefore tend to reduce this number of constraints by relaxing the problem. This, for example, was done in 1960 by Miller *et al.* [MIL 60] by adding an additional variable to reduce the final number of constraints. However, this method yields results that are less optimal than the previous method, although obtained more quickly and for problems with larger dimensions. Rather than using an integer linear program to solve these formulations, algorithms were developed to find or approximate the optimal solution more quickly and with fewer resources. The most widely used

algorithm is the branch-and-bound algorithm, which separates the main problem into subproblems and evaluates the solutions of each subproblem to reconstruct the solution of the initial problem.

5.3.2.2. MTZ formulation

A represents the set of edges (i, j) and V the set of towns, with n being the number of towns to be visited. The value l_{ij} represents the distance between i and j . We also use a binary variable x_{ij} : for each edge $(i, j) \in A$, the variable x_{ij} is 1 if edge (i, j) is taken and 0 otherwise. The quantity U is the visiting rank of i or j and d is the distance traveled [MIL 60]:

$$\min \sum_{i \neq j} d_{ij} x_{ij} \quad [5.10]$$

$$\sum_{j=1}^n x_{ij} = 1 \quad (\forall i \in V, i \neq j) \quad [5.11]$$

$$U_i - U_j + nx_{ij} \leq n - 1, \quad [5.12]$$

$$U_j = U_i + 1 \text{ if } x_{ij} = 1, i \neq j \quad [5.13]$$

$$x_{ij} \in \{0; 1\}, \forall (i, j) \in A \quad [5.14]$$

The constraint [5.10] is the objective function, [5.11] and [5.12] are the assignment constraints, [5.13] is the integrality constraint and [5.14] represents the MTZ subtour elimination constraints. This formulation of the TSP has $(n - 1) \times (n + 1)$ variables and $(n^2 - n + 2)$ constraints.

5.3.2.3. Desrochers–Laporte formulation

This is a reinforced version of the MTZ formulation [DES 91]:

$$\min \sum_{i \neq j} d_{ij} x_{ij} \quad [5.15]$$

$$\text{s.c. } \sum_{j=1}^n x_{ij} = 1 \quad (\forall i \in V, i \neq j) \quad [5.16]$$

$$\sum_{i=1}^n x_{ij} = 1 \quad (\forall j \in V, j \neq i) \quad [5.17]$$

$$U_i - U_j + (n-1)x_{ij} + (n-3)x_{jt} \leq n - 2_{2n} \quad \forall i \neq 1, j \neq 1, \quad [5.18]$$

$$2 \leq U_i \leq n, \quad U_1 = 1 \quad [5.19]$$

$$x_{ij} \in \{0; 1\}, \quad \forall (i, j) \in A \quad [5.20]$$

The constraint [5.15] represents the objective function, [5.16] and [5.17] are assignment constraints, each town is visited exactly once, [5.18] is an integrality constraint and [5.19] is the DL subtour elimination constraint.

5.3.3. Methods for solving the ATSP

Methods for solving the ATSP are generally split into two categories: exact methods and approximate methods (heuristics and metaheuristics). Designed to find the optimum of a problem, exact methods often have computation times that increase exponentially with the size of the problems that they are attempting to solve; this means they must be terminated prematurely during resolution. If so, the exact method only produces a lower or upper bound. By contrast, approximate methods are designed to produce high-quality admissible solutions within a reasonable resolution time, although these solutions are not necessarily optimal. Approximate methods encompass constructive heuristics, local searches that give a sequence of solutions with decreasing (or increasing) costs until the local optimum is attained, and finally metaheuristics that use various mechanisms to escape from local optima. Exact methods and approximate methods are complementary as their results allow them to be evaluated against one another: a small gap between lower and upper bounds means that the optimum is nearby, whereas a gap of zero proves that the values found are optimal. As a result of this complementary relationship between exact methods and approximate methods, one method is often used to improve the performance of another [FIS 94].

Solving an ILP requires the application of dedicated methods such as branch-and-bound methods [LAW 85, NAR 77], branch-and-cut methods [PAD 91, PER 15], column generation methods,

[DES 06, VAN 96], dynamic programming [BEL 56, BER 95] and Lagrangian relaxation [MAC 83].

Exact algorithms are used to find at least one optimal solution to a problem. The most successful exact algorithms in the literature follow the paradigms of four large classes of solving methods:

- dynamic programming;
- constraint programming;
- implicit enumeration methods (branch-and-bound);
- cutting-plane methods.

These methods use bounds to eliminate sets of solutions with a certain structure from the search.

Other methods are specific to certain problems or less general, such as dynamic programming and the cutting-plane method.

5.3.3.1. *Branch-and-bound method*

ATSPs are often solved optimally with the branch-and-bound method, which considers a relaxed problem called the AP [FIS 94]. Instead of having a unique route through every town (node), an optimal solution of the AP generally consists of more than one route, called a subtour. The branch-and-bound algorithm is constructed from the four following steps.

The branching rule specifies how the current problem should be subdivided into subproblems.

The idea is to perform a separation, which requires us to choose a separation variable. This variable is chosen heuristically, with various possible choices that may depend on the effectiveness of the solving method. The pathing strategy through the tree determines the next subproblem that will be considered. There are two widely used strategies, depth-first search (DFS), which starts by solving the last subproblem to be generated, and breadth-first search (BFS), which first solves the subproblem judged to be best, typically the subproblem with the lowest AP bound. The upper bound strategy determines how the routes should be constructed in the branch-and-bound process. One possible method for constructing a feasible route for the ATSP, based on AP, solutions is the readjustment procedure.

The lower bound strategy determines how the lower bound associated with the value of a solution should be constructed for a given subproblem. The value of the AP solution of the subproblem is typically taken as a lower bound. A review of current knowledge about the branch-and-bound algorithm for the ATSP problem can be found in Miller and Pekny (1991). The algorithms

presented by Miller and Carpaneto apply a readjustment to obtain upper bounds using the AP lower bounds, branching on the smallest cycle in the current AP solution. The pathing strategy used by the two algorithms is BFS (best subproblem first), which means solutions are obtained very quickly for many real examples of the ATSP. It also means that a list of subproblems must be maintained to determine the most promising subproblem. Consequently, BFS algorithms tend to encounter memory issues as the pathing tree grows. The upper bounds of the branch-and-bound algorithms for the ATSP were improved by Turkensteen in 2007 with an iterative readjustment, a constructive procedure that yields good feasible solutions for the ATSP. The iterative readjustment procedure allows the number of subproblems in the branch-and-bound tree to be reduced in such a way that the overall computational time is smaller [LAW 85].

Algorithm 5.2 Branch-and-bound algorithm

- 1: **Start**
- 2: Add the starting node of length 0 to a list.
- 3: Repeat
- 4: If the first branch contains the desired node then
- 5: End successfully.
- 6: Else
- 7: Delete the branch from the list and form new branches by extending the deleted branch by one step.
- 8: Compute the cumulative costs of the branches and add them to the list, sorted by increasing order.
- 9: Until (list empty or desired node is found)
- 10: **End**

5.3.3.2. *Cutting-plane method*

The cutting-plane method is an alternative to the branch-and-bound algorithm that is also used to solve integer-valued problems. The fundamental idea is to add suitable constraints called cuts to a linear problem until the optimal realizable basic solution becomes integer valued. Each cut, with respect to a current partial solution, satisfies the following criteria:

- 1) every integer-valued feasible solution is feasible with the cut;
- 2) the current partial solution is not feasible with the cut.

There are various ways to generate cuts. Examples include Gomory cuts, which produces cuts for any table of a linear program. This has the advantage of being applicable to any problem, but the disadvantage of being very slow. The second approach is to use the structure of the problem to produce very high-quality cuts. This requires a case-by-case analysis but can lead to very effective solving techniques.

5.3.3.3. *Branch-and-cut method*

Padberg and Rinaldi [PAD 91] improved the idea of the branch-and-bound algorithm for linear programming (LP) by describing a method that uses inequalities to reinforce LP relaxation (inequalities valid on the convex envelope of integer solutions). They called their method the branch-and-cut method. Since then, many implementations have used this approach.

This method requires the optimization problem to be transformed into an integer LP problem. Unfortunately, it is difficult or even impossible in general to enumerate every inequality that follows from the optimization problem, even if we restrict attention to the inequalities that describe the convex envelope around the optimal region. Once the problem has been transformed, if this is possible, we take advantage of the fact that the real-valued LP problem is easier to solve than the integer-valued problem. The branch-and-cut algorithm uses this specific property to accelerate the search process. The basic principle is the same as the branch-and-bound algorithm, but instead of “branching” on a variable to descend in the search tree, we can instead look for “cutting planes” to restrict the space of feasible solutions. At the start of the algorithm, we restrict attention to a small set of inequalities of the problem that are solved with real numbers. Then, if the optimal solution obtained for the real-valued problem is:

- an integer: we cut the branch being considered, updating the value of the optimal solution if necessary;
- not an integer: we apply a separation algorithm and/or heuristics to extract a set of inequalities that are violated. If this set is empty, we search for an integer solution with this cost, otherwise we update the set of inequalities and continue the algorithm. The goal is to “intelligently” exclude the non-integer optimal solution from the search space using cutting planes.

If the problem being solved does not have a feasible solution, we cut the branch.

There are various ways to generate cutting planes, such as by choosing a linear combination of the inequalities of the LP [GOM 63], using Dantzig's simplex algorithm, or in a manner that is specific to the problem being solved. If we fail to generate a cutting plane and the subproblem cannot be pruned due to the value of the objective, we need to “branch”. The branching operation is performed by specifying a set of hyperplanes that divide the current subproblem in such a way that the current optimal real solution is not feasible in any of the LP relaxation subproblems that are generated (for example, if the optimal real solution x_i is 2.7, we can branch by adding the constraint that $x \leq 2$ in one branch and $x \geq 3$ in the other).

Algorithm 5.3 Branch-and-cut algorithm

/* We want to solve the combinatorial optimization problem $\min \{c^t x : Ax \geq b, x \in \#^n\}$ With $A \in \#^{m \times n}$ and $b \in \#^m$.

List-of-problems = empty;

Initialize the linear program with the subsystem of constraints (A1; b1) with $A_1 \in \#^{m_1 \times n}$ and $b_1 \in \mathbb{R}_1^m$ with $m_1 \ll m$

/* Problem evaluation steps */

Calculate the optimal solution x of the linear program. Current-solution = Apply-themethod-of-polyhedral-cuts ();

/* End evaluation steps */

If Current solution is feasible **then**

$x^* = \bar{x}$ is the optimal solution of $\min \{c^t x : Ax \geq b, x \in \#^n\}$

Else

Add the problem to List-of-problems;

End If

While List-of-problems is not empty **do**

Select a problem;

Branch the problem into two or more subproblems;

Solve the new subproblems (by applying the evaluation steps);

End While

5.4. Vehicle routing problem

5.4.1. Definition

The general problem of constructing routes for vehicles is known as the VRP. This is a multiobjective combinatorial optimization problem that has been widely studied and presented in many different variants in the literature. The VRP was introduced by Dantzig and Ramser, who called it the truck dispatching problem [DAN 59]. It belongs to the category of NP-hard problems [CHR 79, LEN 81, SAV 95].

The principle of the VRP is as follows: given a warehouse D and a set of customer orders $C = \{c_1, \dots, c_n\}$, construct a set of routes for finitely many vehicles that start and end at a warehouse. In these routes, each customer must be served by exactly one vehicle, and the transportation capacity of a vehicle must not be exceeded in a route [NAB 07]. The objective of the VRP is to minimize the sum of the distance traveled or the overall travel time of the vehicles' routes, while satisfying the customer demand.

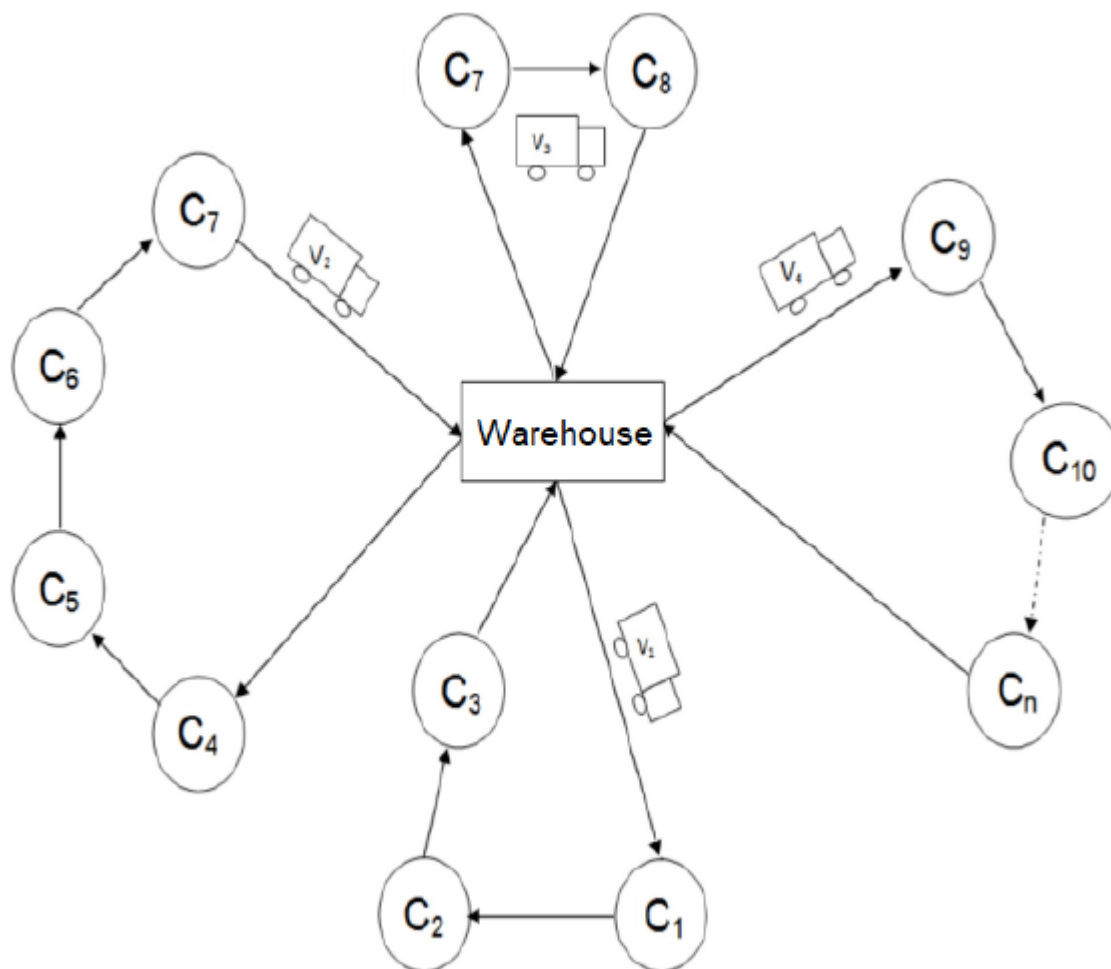


Figure 5.5. VRP

5.4.2. *Fields of application*

The VRP has many diverse applications. Every industrial company wants to improve the efficiency of its logistical chain to reduce the cost of producing goods or services, and to deliver its products smoothly. The VRP is a key link in the field of logistics. Given the massive increase in the cost of energy, almost 30% of the cost of producing a product or a service is due to transportation.

The VRP is part of our everyday lives, starting with picking schoolchildren up, picking up employees, collecting household waste, harvesting raw milk, distributing newspapers, mail and food products such as milk, bread and water. Ambulance services and urgent distribution of pharmaceutical products are also examples of the VRP.

In the last two examples, customers are served according to priority and urgency, and the objective is not to minimize the total distance traveled but to serve as many clients as possible in a given time frame. Note that all of these problems are constrained by time, and collection problems have more rigid time constraints than the others, which are distribution problems.

Information technology and robotics are other fields in which the VRP is widely used. Indeed, routing information in remotely connected computer networks or over wires is fundamentally based on the VRP. Aircraft scheduling is another example of an application.

Despite the interest that people have shown in this problem, to this day we have been incapable of solving some instances of it in practice. Large-scale instances of the VRP have proven to be difficult or complex and sometimes even “impossible” to solve with a calculator, regardless of its computational power.

5.4.3. *Parameters of the VRP*

The VRP is characterized by the network of the problem, the clientele that needs to be served, the fleet of vehicles and the objective function:

- *Network*: the transportation network allows the flow of individuals, freight or information.

In a certain sense, it provides the skeleton of a system that aims to establish some form of communication. It can be represented as a complete asymmetric or symmetric graph. The vertices represent the clients, characterized by their geographical positions (the coordinates (x, y)) and the edges represent the paths between clients. With some constraints, this graph can be directed.

- *Clientele*: each client is characterized by a demand for a service or products (goods) of one or several types. The total demand of the clients on a single route must not exceed the capacity Q of the vehicle. Clients are also characterized by a position in space. Finally, the demand can be deterministic (fixed quantity demanded by the client that is known by the distributor) or uncertain (stochastic).
- *Fleet of vehicles*: the first criterion of the fleet is its size (number of vehicles), the second is its homogeneity (the vehicles are characterized by the same capacity and transportation cost) or heterogeneity (the vehicles have different capacities and/or transportation costs).
- *Objective function*: the most common objectives are to minimize the number of vehicles used or minimize the total distance traveled by the vehicles.

Other objectives can also be considered:

- minimizing the total time taken to complete the route, the waiting time, the delay, the service time, etc.;
- minimizing the number of vehicles;
- minimizing the total cost of the route. In the fixed cost case, we have amortization of equipment (vehicles and otherwise), driver salaries, vehicle fees, etc. In variable cost problems, late penalties (especially in the VRP-TW), etc.;
- maximizing the service quality;
- maximizing loading of the vehicles on routes.

5.4.4. Variants of the VRP

Several variants of the VRP can be defined by varying its parameters or by deleting, adding and/or combining the constraints of the classical problem. A classification of the variants of the VRP according to type of constraint is as follows:

- VRPC (vehicle routing problem with capacities): this is a routing problem with capacity constraints. The vehicles have a limited capacity (quantity, volume, weight, etc.). This is translated by the constraint that the sum of client demands on a given route must not exceed the vehicle capacity. In practice, every VRP is a limited capacity problem; only the extent of the influence of this constraint varies. For example, problems involving the transportation of goods are very sensitive to capacity, whereas the mail distribution problem is less sensitive.

- VRPCC (vehicle routing problem with complete capacity): this is a routing problem that requires the vehicle's capacity to be completely utilized.
- VRPHF (vehicle routing problem with heterogeneous fleet): the fleet is made up of vehicles of different types that are distinguished by their capacity, power, transportation cost, etc.
- OVRP (open vehicle routing problem): identical to the VRP except the vehicle has a free choice of whether to return to the warehouse after completing its route. If the vehicle chooses to return to the warehouse, it must travel the same route in reverse.
- RVRP (return vehicle routing problem): in this problem, the vehicle is required to return to the warehouse. The vehicle must return to the warehouse immediately after serving the last client, without needing to repeat its route.
- VRP with deterministic demand: frequently encountered by companies that deliver on request. The delivery driver knows the quantity of goods that must be delivered to each client before leaving the warehouse.
- VRP with stochastic demand: unlike the previous problem, the delivery driver does not know the quantity of goods to deliver to each client. This quantity is only revealed at the time of delivery. The demand of each client is estimated approximately with a stochastic function.
- SVRP (split delivery vehicle routing problem): involves visiting clients several times to fully satisfy demand. An exception for this problem, the client's demand can exceed the vehicle capacity.
- VRPCD (vehicle routing problem with collection and delivery): this problem extends the VRP to both collection and delivery. With this type of problem, the service time is counted twice, since both a collection and a delivery must be performed or vice versa.
- VRPMW (vehicle routing problem with multiple warehouses): vehicles can be supplied by multiple warehouses.
- VRP1W: vehicles are supplied by a single warehouse.
- VRPMP (vehicle routing problem with multiple products): a range of products must be delivered to different clients by each vehicle in a single route.

- VRP (vehicle routing problem with a single product): a single product must be delivered to different clients by each vehicle in a single route.
- PVRP (periodic vehicle routing problem): this problem has it that each client is periodically visited according to a predefined schedule.
- VRP with deterministic service time: the service time is known by the delivery driver before starting the route.
- VRP with stochastic service time: the delivery driver does not know the service time, which is only revealed when clients are served. A stochastic function is defined to approximate the service time.
- VRP-TW (vehicle routing problem with time windows): this extends the VRP to include time windows. Each client must be served in a certain time window that is known to the delivery driver in advance, and any violation of this constraint may incur a penalty. If the time window constraint is not satisfied, either the solution is rejected in the rigid version of the problem, or a penalty function is constructed and added or otherwise combined with the objective function in the relaxed case. In practice, this is a very common problem. The distribution of perishable products (milk, meat, etc.), newspapers, ambulance services, etc., are practical examples of the VRP-TW. In this class of problems, we can distinguish between two subclasses: the rigid VRP-TW, where the service must necessarily be provided within the time window, and the relaxed VRP-TW, where only being early or late incurs a penalty. Adding this penalty defines a variant of the VRP-TW.
- Static VRP: a problem where every component is known and fixed before beginning any route. This case is obtained either by a rigorous stationarization study of the set of parameters or because the problem being modeled describes a static phenomenon.
- Dynamic VRP: as opposed to the static VRP, the dynamic VRP has at least one dynamic or variable component at the time of execution. For example, the demand or the service time window might vary, or the number of clients to be served might change. This makes the problem more complex.
- SVRP (stochastic vehicle routing problem): this represents a special case of the dynamic VRP where the variable component follows a probabilistic law whose variation is described by a mathematical model, allowing this variation to be manipulated, making the problem easier to solve.

5.4.5. Mathematical formulation of the VRP

There are various formulations of the VRP in the literature. An aspect shared by all of them is that the VRP is represented as a directed graph. The graphical representation of the classical VRP is as follows: let $G = (X, U)$ be a directed graph, where:

- X is the set of vertices of the graph, G is the set of clients and warehouse(s) of the VRP, $X = N \cup \{0\}$, and $(|X| = n+1)$; U is the set of edges of the graph, G represents the paths between clients and between clients and warehouses, $(|U| = \frac{(n+1)n}{2})$;
- weights can be assigned to the vertices (or edges) to define the quantities demanded by clients (or the distance separating two clients, or the displacement time, etc.).

There are many mathematical formulations of the classical VRP in the literature. The two formulations presented below are among the most commonly used. First, we must define the following parameters:

- n : number of clients; m : number of vehicles ($m \leq n$);
- Q_k : capacity of the k th vehicle; T_k : maximum time of the route of the k th vehicle;
- as_j : number of edges leaving the vertex j ; ae_j : number of edges entering the vertex j ;
- q_j : demand of the j th client $d_0 = 0$;
- sk_j : service time of the j th client by the k th vehicle ($sk_0 = 0$);
- t_{ij}^k : time needed to go from the i th client to the j th client ($t_{jj}^k = +\infty$);
- d_{ij}^k : distance between the i th client from the j th client ($d_{jj}^k = +\infty$);
- C_{ij} : cost of moving from the i th client to the j th client.

Formulation

The most widely used formulation of the classical VRP in the literature is the formulation adopted by Laporte [LAP 92], Toth and Vigo [TOT 01], and Crainic and Semet [CRA 14]. It requires us to define $n \times n$ binary decision variables with the three following indices:

$$x_{ij}^k = \begin{cases} 1 & \text{if the edge (i,j) is traveled by the } k\text{-th vehicle,} \\ 0 & \text{otherwise,} \end{cases} \quad [5.21]$$

In other words: $x_{ij}^k \in \{0, 1\}$; $i = 0, \dots, n$, $j = 0, \dots, n$, $k = 0, \dots, m$.

The problem is modeled as follows:

$$\min \sum_{i=1}^n \sum_{j=1}^n c_{ij} \sum_{k=1}^m x_{ij}^k \quad [5.22]$$

$$\sum_{i=1}^n \sum_{k=1}^m x_{ij}^k, \quad j = 1, \dots, n \quad [5.23]$$

$$\sum_{j=1}^n \sum_{k=1}^m x_{ij}^k, \quad i = 1, \dots, n \quad [5.24]$$

$$\sum_{i=1}^n x_{ijp}^k - \sum_{k=1}^m x_{ipj}^k = 0; \quad k = 1, \dots, m, \quad p = 1, \dots, m \quad [5.25]$$

$$\sum_{i=1}^n q_i \left(\sum_{j=1}^n x_{ij}^k \right) \leq Q_k, \quad k = 1, \dots, m \quad [5.26]$$

$$\sum_{j=1}^n s_i^k x_{ij}^k + \sum_{i=1}^n \sum_{j=1}^n t_{ij}^k x_{ij}^k \leq T_k, \quad k = 1, \dots, m \quad [5.27]$$

$$\sum_{j=1}^n x_{0j}^k \leq 1, \quad k = 1, \dots, m \quad [5.28]$$

$$\sum_{i=1}^n x_{i0}^k \leq 1, \quad k = 1, \dots, m \quad [5.29]$$

$$\sum_{i \in S} \sum_{j \in S} x_{ij}^k \leq |S| - 1, \quad \forall S \subset N \text{ et } 2 \leq |S| \leq (n - 1). \quad [5.30]$$

Formula [\[5.22\]](#) represents the function to be optimized or the objective function of the classical VRP. In general, the goal is to find the minimum of the global routing cost. Formulas [\[5.23\]](#)–[\[5.30\]](#) represent the constraints of the problem.

The constraints [\[5.23\]](#) and [\[5.24\]](#) guarantee that each client is only served once and only by a single vehicle. The constraint [\[5.25\]](#) guarantees the continuity of the route: not only must each vehicle pass exactly once through a client, [\[5.24\]](#) and [\[5.25\]](#), the vehicle must necessarily also leave this client once the service is complete.

The constraint [\[5.27\]](#) guarantees that the capacity of the vehicle is not exceeded. The constraint [\[5.28\]](#) guarantees that the total duration of a route never exceeds its total maximum duration.

The constraints [5.28] and [5.29] guarantee that vehicle availability is not violated by ensuring that each vehicle only leaves and returns to the warehouse once. The final constraint [5.30] eliminates any subtours.

5.4.6. Algorithmic complexity

Evaluating the complexity of an arbitrary problem means estimating the number of instructions that must be performed in the worst-case scenario, to solve an instance of the problem. The algorithmic complexity measures the algorithm's efficiency at solving the problem. The higher the number of instructions, the more complex the problem. Thus, the algorithmic complexity is proportional to the number of instructions in the problem. Similarly, the more complex the algorithm, the less efficient it is.

We can distinguish between several types of complexity, such as constant complexity $O(1)$, which is independent of the problem size, logarithmic complexity $O(\ln(n))$, linear complexity $O(n)$, quadratic complexity $O(n^2)$, cubic complexity $O(n^3)$, polynomial complexity $O(n^p)$, exponential complexity, and finally factorial complexity $O(n!)$.

The complexity criterion allows us to distinguish between four classes of problems as follows:

- P class: this class contains all problems that can be solved in polynomial time by a Turing machine. Solving any instance of a problem in this class can be done in polynomial time. This class is known as the class of polynomial problems or P class.
- P -complete class: this class contains all polynomial problems and non-polynomial problems that can be reduced in poly-logarithmic time with or without the use of a deterministic calculating machine.
- NP class: this class contains all problems that can be solved in polynomial time on a Turing machine.
- NP -hard class: this class contains all problems whose complexity is exponential or doubly exponential. It is characterized by the difficulty of solving large problem instances exactly or optimally in polynomial time. Instead, we must be satisfied with so-called high-quality solutions in polynomial time. The problems of this class include tree partitioning problems, connectedness problems in a routing graph, etc.

The VRP is a routing problem. Lenstra and Rinnooy Kan [LEN 81] showed that the VRP is NP -hard. Consequently, it is not appropriate to attempt to solve large problem instances with exact

methods. Instead, we must solve these with heuristic approaches that yield feasible and useful solutions in a reasonable time frame.

5.5. Selective routing problem

Faced with the current increase in energy prices, the transportation sector is showing a growing interest in this problem. Indeed, to minimize transportation costs, companies regularly seek to streamline their logistics systems to remain competitive. Logistics systems consist of various elements, one of the most important of which is the vehicle routing schedule in their transportation networks.

Over the past century, several models and methods have been proposed to answer the various problems encountered in logistics systems. These techniques are directly linked to the evolution of computer systems. Today, in just a few seconds, with a calculator of average capacity, we can solve logistics problems that would have been completely impossible to solve historically.

For example, we can study problems involving the distribution of goods between warehouses and clients. These problems are known as VRPs. The VRP was introduced by Dantzig and Ramser, who called it the truck dispatching problem [DAN 54]. It involves planning the routes of a fleet of vehicles to serve a set of clients. Constructing the routes becomes more complex after constraints and objectives are added to the problem. Some of the most important examples include maximizing client satisfaction and minimizing the cost of the routes.

The selective vehicle routing problem (SVRP) is another problem derived from the VRP. The SVRP is one variant of a set of problems in a class called vehicle routing problems with profits (VRPPs) [FEI 01]. VRPPs attempt to construct one or several routes that maximize the total profit generated from clients, while minimizing the route length. By their nature, these problems are dual-objective problems.

5.5.1. Problems similar to the VRP

There are three classes of problems similar to the VRP [FEI 01] as follows:

- selective traveling salesman problem (STSP): the goal is to find a route that maximizes the total profit subject to an upper bound on the cost of the route;
- prize-collecting traveling salesman problem (PCTSP): the goal is to find a route of minimum cost subject to a lower bound on the total profit of the route;

– profitable tour problem (PTP): the goal is to find a route that optimizes a linear combination of the two previous objectives. Note that the notion of cost of a route often reduces to the duration or total length of the route, which is the sum of the travel times or lengths of the arcs that make up this route.

The orienteering problem (OP) is another well-known problem that is similar to the STSP.

5.5.2. *Mathematical formulation*

The earliest description of the STSP can be found in [GOL 87]. The authors consider a periodic vehicle routing problem that accounts for stock. A fleet of vehicles must deliver fuel to a large number of clients every day. The problem is modeled as an STSP where the profit measures the urgency. In the early 1980s, there was also a description of a similar problem, called the OP [TSI 84]. This problem was inspired by a sport practiced outdoors in mountainous or forested regions. Competitors with maps are challenged to travel from a starting point to a certain destination, within a fixed time period, while passing through certain checkpoints. A score is awarded for each checkpoint, and the goal is to maximize the total score. The STSP is in fact a special case of the OP where the starting point and the destination coincide. Some generalizations of the OP have also been discussed in the literature. Thus, Kantor and Rosenwein [KAN 92] presented the OP with time windows, where the checkpoints can only be visited within certain windows. In [CHA 96], the authors extended the problem to the case of a team consisting of several competitors.

Like the OP and the PTP, the STSP belongs to the class of *NP*-complete problems. Indeed, these problems are generalizations of the TSP, which is itself *NP*-complete. Below, we give an integer-valued formulation of the oriented version of the STSP. Let $G = (N, A)$ be a complete graph, where $N = \{1, \dots, n\}$ is the set of n vertices of the graph. Vertex 1 plays the role of the warehouse, and A is the set of edges. We assign a profit g_i to each vertex $i \in N$ (with $g_1 = 0$) and a cost c_{ij} to each edge $(i, j) \in A$ (the cost is often the length of the edge).

The STSP asks us to find an elementary cycle of maximum profit that contains vertex 1 and whose cost does not exceed some bound $C_{\max}$.

The formulation presented in [FEI 01] assigns a binary variable x_{ij} to each edge $(i, j) \in A$, where x_{ij} is equal to 1 if the corresponding edge is used in the solution and 0 otherwise. A binary variable

y_i is also assigned to each vertex $i \in N$, where y_i is equal to 1 if the corresponding vertex is visited and 0 otherwise. Using these variables, the STSP can be formulated as the following integer program:

$$\max \sum_{i \in N} y_i p_i \quad [5.31]$$

$$\text{s.t.} \quad \sum_{j \in N - \{i\}} x_{ij} = y_i, \quad i \in N \quad [5.32]$$

$$\sum_{i \in N - \{j\}} x_{ij} = y_j, \quad j \in N \quad [5.34]$$

$$\sum_{i \in N} \sum_{j \in N - \{i\}} C_{ij} x_{ij} \leq C_{\max} \quad [5.36]$$

$$x_{ij} \in \{0, 1\} \quad [5.38]$$

$$y_i \in \{0, 1\} \quad [5.40]$$

The constraints [5.33] and [5.35] are the assignment constraints. The constraint [5.37] [5.39] imposes a maximum cost. The STSP can be solved using heuristics, metaheuristics or [5.41] exact methods.

5.6. Using Matlab

Matlab documentation includes a (lengthy) script “TravellingSalesmanExample.m” on the TSP. [Figure 5.6](#), generated by this script, is shown below.

As noted above, as an alternative, we could use a genetic algorithm to solve the problem. We will return to this type of algorithm in a subsequent volume to show how they can be used to solve a TSP.

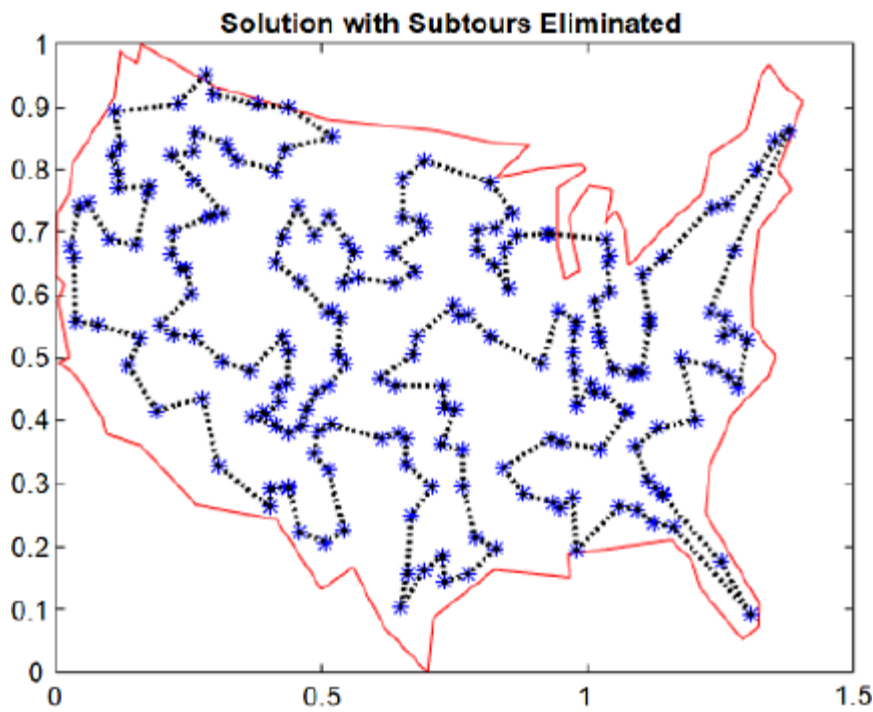


Figure 5.6. Final tour of the traveling salesman. For a color version of this figure, see www.iste.co.uk/radi/optimizations.zip